ELSEVIER

Reliability Engineering and System Safety 67 (2000) 215–232

RELIABILITY ENGINEERING & SYSTEM SAFETY

www.elsevier.com/locate/ress

Constrained optimization of test intervals using a steady-state genetic algorithm

S. Martorell*, S. Carlos, A. Sánchez, V. Serradell

Departamento de Ingeniería Química y Nuclear, Universidad Politécnica de Valencia, P.O. Box 22012, 46071 Valencia, Spain

Received 19 October 1998; accepted 5 November 1999

Abstract

There is a growing interest from both the regulatory authorities and the nuclear industry to stimulate the use of Probabilistic Risk Analysis (PRA) for risk-informed applications at Nuclear Power Plants (NPPs). Nowadays, special attention is being paid on analyzing plant-specific changes to Test Intervals (TIs) within the Technical Specifications (TSs) of NPPs and it seems to be a consensus on the need of making these requirements more risk-effective and less costly. Resource versus risk-control effectiveness principles formally enters in optimization problems. This paper presents an approach for using the PRA models in conducting the constrained optimization of TIs based on a steady-state genetic algorithm (SSGA) where the cost or the burden is to be minimized while the risk or performance is constrained to be at a given level, or vice versa. The paper encompasses first with the problem formulation, where the objective function and constraints that apply in the constrained optimization of TIs based on risk and cost models at system level are derived. Next, the foundation of the optimizer is given, which is derived by customizing a SSGA in order to allow optimizing TIs under constraints. Also, a case study is performed using this approach, which shows the benefits of adopting both PRA models and genetic algorithms, in particular for the constrained optimization of TIs, although it is also expected a great benefit of using this approach to solve other engineering optimization problems. However, care must be taken in using genetic algorithms in constrained optimization problems as it is concluded in this paper. © 2000 Elsevier Science Ltd. All rights reserved.

Keywords: Technical specifications; Test intervals; Safety-related systems availability; Probabilistic risk assessment; Risk-informed decision-making; Risk-consistency check; Cost and burden; Resource-effectiveness; Constrained optimization; Genetic algorithms

1. Introduction

1.1. Risk-informed applications

It is a widely recognized growing interest from both the regulatory authorities and the nuclear industry to stimulate the use of Probabilistic Risk Analysis (PRA) for risk-informed application aimed to emphasize both effective risk control and effective resource expenditures at Nuclear Power Plants (NPPs) [1–6]. The general goal of these risk-informed applications is to make requirements and activities more risk-effective and less costly by using the risk information to focus better on what is risk-important [7]. Risk-informed applications incorporate broader information than risk-based applications as the former include both risk and deterministic information into decision-making.

However, it is also recognized that there has not been any widespread response until now. In addition, the main focus has been on identifying contributions of small risk importance so that relaxing requirements on them results in both reducing burdens and a small or negligible risk increase. There has been little focus on the most interesting risk-informed applications aimed to reduce burdens while controlling risk by improving effectiveness. Therefore, the focus on most of the current risk-informed applications seems to be far away from its original aim as discussed above and in detail in Refs. [1,7].

There are several activities in plant associated with controlling risk or with satisfying requirements, such as Technical Specification (TS), In-Service Test (IST), In-Service Inspection (ISI), Preventive Maintenance (PM), etc., which are candidate to be evaluated for their resource effectiveness in risk-informed applications. Nowadays, special attention is being paid on the use of the PRA for risk-informed decision-making on plant-specific changes to test intervals in TSs [8,9].

* Corresponding author. Tel.: +34-96-3877098; fax: +34-96-3877099.
E-mail address: smartore@iqn.upv.es (S. Martorell).

0951-8320/00/$ - see front matter © 2000 Elsevier Science Ltd. All rights reserved.
PII: S0951-8320(99)00074-5

1.2. Optimizing effectiveness of test intervals

The resource versus risk-control effectiveness principles formally enter optimization problems where the cost or the burden (i.e. number of test conducted, man-hours required, incurred costs, occupational exposure, etc.) is to be minimized while the risk or performance is constrained to be at a given level. This relationship applies when test intervals requirements within TSs are optimized to minimize the cost while controlling the unavailability of safety-related systems and plant risk, as suggested in Refs. [10–12]. Alternatively, the same relationship also applies when unavailability or risk is to be minimized for a given resource expenditure taking test intervals as the decision variables.

As suggested in Ref. [10], the optimization problem can be formulated in terms of an objective function, to be either minimized or maximized, under constraints that apply for a given scope (i.e. component, system or plant performance) and the method adopted to perform the parameter optimization (the optimizer). In fact, in optimization, objective functions and constraints cannot be handled independently of the underlying optimizer.

In optimizing test intervals based on risk (or unavailability) and cost, like in many engineering optimization problems (i.e. design, reliability, etc.), one normally faces multi-modal and non-linear objective functions and a variety of both linear and non-linear constraints. This results in a complex and discrete search space with regions of feasible and unfeasible solutions for a discontinuous objective function that eventually presents local optima.

For such problems, it is desired to find global optima not violating any constraint. In addition, requirements such as continuity and differentiability of both objective and constraints functions add yet another conflicting element to the decision process. As discussed in Refs. [10–12], apart from very simplified problems, resolution of such optimization problems requires numerical methods [11,13–15]. However, as traditional approaches usually give poor results under these circumstances, new methods based on Genetic Algorithms (GAs) were investigated in order to try to solve this kind of problems [16,17], because GAs possess many characteristics desirable in such a kind of optimizer. An optimization method based on GAs was successfully investigated in Ref. [10] for optimizing surveillance testing and maintenance of components. The scope of the optimization problem is extended in this paper at system level considering additional constraints, which requires a genetic algorithm with extended capabilities.

1.3. Brief overview of GAs in engineering optimization problems

GAs are very likely to be the most widely known type of Evolutionary Algorithms (EAs) [18]. GAs are adaptive methods that can be used in searching and optimization problems which work by imitating the principles of natural selection and genetics. GAs in their usual form were developed by John Holland in 1965 who summarized his work in Ref. [19]. His work and a number of PhD dissertations by his students [20] were the only documents available on GAs until late 1970s, which serve as starting point of nearly all known applications and implementations of GAs.

Owing to their simplicity, flexibility, easy operation, minimal requirements and global perspective, GAs have been successfully used in a wide variety of problems in several areas of engineering and life science, as described for example in Refs. [17,21–22]. Additionally, in the recent years an increasing number of GAs applications in the fields of optimizing equipment reliability, maintainability and availability are being observed [10,12,23–25]. Fundamentals on how GAs work, GAs implementation and their applications are introduced in many papers [16–17,21] and are described in detail in several books [18,26–29].

EAs are essentially unconstrained search techniques, which require the assignment of a scalar measure of quality, or fitness, to candidate solutions to the optimization problem. In fact, GAs are widely used now for unconstrained optimization problems with the general agreement about the fundamentals of the implementation as can be observed in the references mentioned above. However, in the last few years, there has been a growing effort to apply GAs to general constrained optimization problems since most of engineering optimization problems often present their solution constrained by a number of restrictions imposed on the decision variables [30–35]. This is the general case to be considered in this paper, as described previously.

In engineering optimization problems, constraints can be classified as explicit or implicit [32,33]. The former involves, for example, the parameter to be optimized (decision variables) that can only adopt a given set of values, where violation is directly revealed. However, an implicit constraint requires a simulation to check for a violation. For example, it is necessary to evaluate first the cost or risk associated with given values of the decision variables to reveal a violation on either of them.

Many approaches have been considered for handling solutions that violate one or more constraints, but no one general method has emerged. Approaches can be categorized as follows [30–33]: (1) eliminating infeasible solutions; (2) repairing infeasible solutions; (3) using modified genetic operators; and (4) applying penalties for infeasible solutions. The first approach is largely used, as for example in Refs. [10,25–26], but it gives poor results particularly in highly constrained problems. Repairing infeasible solutions and using modified genetic operators are more useful for handling explicit constraints since handling implicit constraints is more difficult an usually too time consuming. The last approach is the most prevalent technique for handling implicit constraints although it is also possible to handle explicit constraints. This technique transforms the

constrained problem into an unconstrained problem by penalizing those solutions that are infeasible after evolution.

The GAs software for this work uses the GAlib genetic algorithm package, written by Matthew Wall at the Massachusetts Institute of Technology. This software is highly customizable allowing the user to implement its own GA-based optimization approach by selecting from a wide library of available GAs that covers most of the currently used methods.

A Steady-State GA (SSGA) has been customized to perform the optimization problem using GAlib. Both binary and real-valued encoding of parameters are possible. The last has been preferred in this paper since problem formulation of the objective function and explicit constraints can be introduced in a very simple and natural way. In addition, new routines have been developed and implemented, mainly for handling implicit constraints using the penalization approach and also for verifying the GAs termination criteria.

This paper encompasses in Section 2 the problem formulation, where the objective function is derived and constraints that apply in the constrained optimization of test intervals based on risk and cost functions at system level. Fundamentals of SSGA as optimization method is given in Section 3, which satisfies the above requirements, paying special attention to its use in constrained optimization problems. A simple case of application is provided in Section 4, focussing on test interval optimization for a stand-by safety-related system, which demonstrates how the SSGA-based optimization approach works at system level. Finally, Section 5 presents our conclusions and additional remarks.

2. Problem formulation

It follows from the introduction that the objective of this paper is focused on optimizing effectiveness of Test Intervals (TIs) of safety-related system components based on risk and cost criteria. As it will be shown in this section, TIs are represented through appropriate parameters included within the models of risk and cost, which will be adopted as decision variables for the optimization process. Alternatively, many other parameters in the models may be considered as decision variables, as one can realize in reviewing previous work in this field, for example, considering optimization of parameters concerning component reliability characteristics, maintenance activities, and so on [10–12,14,25]. Therefore, from now on, the variable x_i will be used with a general meaning, representing a given parameter, i, in the model that may be adopted as a decision variable for the optimization, while the variable T will be adopted to represent a particular test interval.

2.1. Objective function and constraints

The optimization problem can be formulated in terms of an objective function for a given scope, where the optimizer is intended to find the solution constrained by a number of restrictions imposed on the decision variables.

Constraint satisfaction and multi-objective optimization can be seen very much two aspects of the same problem. Both involve the simultaneous optimization of a number of functions. Thus, constraints can often be seen as hard objectives, which need to be satisfied before the optimization of the remaining, soft, objective takes place. Conversely, problems characterized by a number of soft objectives are often re-formulated as constrained optimization problems in order to be solved.

The second alternative has been chosen in this paper. The constrained optimization of TIs will then be formulated by adopting either the risk or cost as the objective function to be minimized while the remaining function, cost or risk, will be considered as an additional restriction belonging to the set of constraints imposed on the decision variables.

Constrained optimization can be represented as a non-linear programming problem [29–31]. The general non-linear programming problem can be defined in terms of an objective function to be minimized:

$$f(\mathbf{x}) \tag{1}$$

subject to non-linear and linear constraints, which is a real-valued function of the decision variable vector $\mathbf{x} = \{x_1, \ldots, x_i, \ldots, x_n\} \in \mathbb{R}^n$ representing the parameters to be optimized. In addition, usually constraints can be expressed in terms of function inequalities of the type:

$$g(\mathbf{x}) \leq g \tag{2}$$

where $g(\mathbf{x})$ generally is a non-linear real-valued function of the decision variable vector $\mathbf{x}$, and g is a constant value. In particular, the inequality may also be strict ($<$), or alternatively, equality ($=$) may be formulated as a particular case of inequality constraints. Finally, in many cases the symbol ‘ $\leq$ ‘ in the above Eq. (2) may be replaced alternatively by symbol ‘ $\geq$ ‘ to formulate the appropriate constraints.

In summary, without loss of generality, the constrained optimization problem may be that of minimizing, alternatively maximizing, a scalar function $f(\mathbf{x})$ of some decision variable vector $\mathbf{x}$ in a universe subject to a number m of conditions involving $\mathbf{x}$ and eventually expressed as a functional vector inequality of the type:

$$(g_1(\mathbf{x}), \ldots, g_m(\mathbf{x})) \leq (g_1, \ldots, g_m) \tag{3}$$

where the inequality applies component-by-component.

2.2. Risk function

The Probabilistic Risk Assessment (PRA) of a Nuclear Power Plant (NPP), both models and methods, provides a quantitative representation of the risk level of the plant through the assessment of the probability of accidents and their consequences. Some of the factors that influence or are

incorporated in this modeling and quantification process are the TIs requirements.

The PRA model structure can be seen as a function of design and TS [34], the last including TIs. Alternatively, it can be seen as a function of the unavailability of the associated safety functions that are intended to prevent the occurrence of accidents or mitigate their consequences. Unavailability of a safety function, in turn, depends on the unavailability of the associated safety-related systems, normally on standby and ready to operate on demand, that performs the corresponding safety function. Finally, the unavailability of safety systems depends on human errors and the unavailability of the components that constitute the system, the last related to random failures, common cause failures and test and maintenance downtimes, which, in turn, depend on TS.

As stated in Section 1, herein, the scope of the problem concerning TIs optimization is limited at system level; therefore, the corresponding system unavailability model in the PRA will be adopted to represent the risk function. It is assumed that by optimizing TIs based on minimizing the corresponding safety-related system unavailability one should improve the NPP safety level.

A structure function is a useful tool in describing the way several components are related to form a system [35]. The structure function, named herein $\Phi(i, \mathbf{x})$, defines the system state (available or unavailable) as a function of the component states. Normally, Fault Trees (FTs) are used to represent such structure function into the PRA model, in which the unavailability state of the system is named the 'top event' while the different unavailability states of the safety components are named 'basic events'. Then, one can derive the system unavailability model by quantifying the probability that the system is found in the downed state, which can be formulated as follows:

$$U(i, \mathbf{x}) = \Pr\{\Phi(i, \mathbf{x}) = 0\} \tag{4}$$

representing $U(i, \mathbf{x})$, in short $U(\mathbf{x})$, the probability that the system is found unavailable (down or 0 state) to perform its intended safety function after the occurrence of an initiating event, named i, corresponding to an accidental sequence in the PRA model. Considering only coherent systems, quantification of the system unavailability is possible using several methods [35,36]. Two of the most common procedures include Minimal Cut Sets (MCS) determination and the Monte Carlo simulation. The former has been chosen herein as it allows showing in a very simple way the relevant dependencies on $\mathbf{x}$.

System unavailability quantification based on MCS involves first the reduction of the coherent system structure function, $\Phi(i, \mathbf{x})$, to yield a series arrangement of parallel subsystems, the last representing each a MCS. In turn, every MCS represents a minimum set of unavailability states of the safety components (basic events in the PRA) in such a way that the system holds the unavailability state if, and only if, all the basic events in the cut set occur without exceptions. Correspondingly, the system unavailability model is often formulated into the PSA adopting the rare-event approximation, which is an upper bound, as follows [36]:

$$U(\mathbf{x}) \approx \sum_j \prod_k u_{jk}(\mathbf{x}) \tag{5}$$

where the sum in j extents to the number of MCS after the reduction of the system structure function and the product in k extents to the number of basic events relevant to the corresponding MCS. In addition, $u_{jk}(\mathbf{x})$ represents the unavailability associated with the basic event k belonging to MCS number j, which can quantify a given unavailability state of a safety component that depends on the vector of decision variables $\mathbf{x}$.

Consequently, quantification of the different contributions to the component unavailability is needed to derive the final risk function using the model in Eq. (5). Several models are suggested in the literature to account for these contributions depending on the type of component. As most of the safety-related components are normally in standby, and TIs affects mainly that kind of components, the corresponding models will be considered herein. Special attention must be given on those models that explicitly account for the parameters in the vector of decision variables, in particular TIs in this case of optimization, as it is done, for example, in Refs. [34,37–41].

In general, the unavailability contributions of a component normally in standby can be divided into two main categories: (1) unavailability due to random failures, named the reliability effect; and (2) unavailability due to testing and maintenance downtimes, named the downtime effect.

Regarding the models to account for the reliability effect, the following expression is normally used [34,37–38]:

$$u_r(\mathbf{x}, \tau) = \rho + (1 - \rho)\cdot\left(1 - e^{-\int_0^{\tau}\lambda(w)\,dw}\right) \tag{6}$$

which represents a time-dependent (τ) unavailability contribution due to random failures, where ρ represents a cyclic or per-demand failure probability and $\lambda(w)$ represents the standby failure rate, which, in general, can be assumed depending on the component age, w. Age-dependent models for $\lambda(w)$ considering the effects of maintenance and working conditions are presented in Refs. [39,40]. An averaged contribution over a given period, T, is derived by integrating Eq. (6) as follows:

$$u_r(\mathbf{x}) = \frac{1}{T}\cdot\int_0^T u_r(\mathbf{x}, \tau)\,d\tau \tag{7}$$

where T will represent the TI of the component in this case of application.

Normally, it is assumed a constant standby failure rate, $\lambda(w) = \lambda$ in PRA models, where, in addition, the following

restrictions usually apply:

$$\left.\begin{array}{l}\rho < 0.1\\ \lambda\cdot\tau < 0.1\\ \lambda\cdot T < 0.1\end{array}\right\} \quad (8)$$

which simplify Eqs. (6) and (7) to yield two common expressions in PRA models [34]:

$$u_r(\mathbf{x}, \tau) \approx \rho + \lambda\cdot\tau \quad (9)$$

$$u_r(\mathbf{x}) \approx \rho + \frac{1}{2}\lambda\cdot T \quad (10)$$

Regarding the models to account for the downtime effect, the following general expressions are commonly used in standard PRA [34,39]:

$$u_t(\mathbf{x}) = f_t(\mathbf{x})\cdot t\cdot q_0^t \quad (11)$$

$$u_m(\mathbf{x}) = f_m(\mathbf{x})\cdot m\cdot q_0^m \quad (12)$$

$$u_c(\mathbf{x}) = f_c(\mathbf{x})\cdot d \quad (13)$$

where $u_t(\mathbf{x})$ represents the unavailability contribution due to testing, $u_m(\mathbf{x})$ is the unavailability contribution due to performing preventive maintenance and $u_c(\mathbf{x})$ is the unavailability contribution due to performing corrective maintenance. In addition, the following notation has been used:

$f_t(\mathbf{x})$ = the rate of testing events;
$f_m(\mathbf{x})$ = the rate of preventive maintenance events;
$f_c(\mathbf{x})$ = the rate of corrective maintenance events;
t = mean downtime due to testing;
m = mean downtime due to preventive maintenance;
d = mean downtime due to corrective maintenance;
q_0^t = fraction of the total downtime t with the component unavailable [0, 1];
q_0^m = fraction of the total downtime m with the component unavailable [0, 1].

An illustrative example corresponds to the case of a safety component that undertakes scheduled time-directed preventive maintenance with periodicity M, where $f_m(\mathbf{x}) = 1/M$, and surveillance testing with Test Interval T, where $f_t(\mathbf{x}) = 1/T$. Formulation of $f_c(\mathbf{x})$ is a little more complex, but for a component normally in standby which undertakes surveillance testing each T, one can adopt the following approximation [34]:

$$f_c(\mathbf{x}) \approx \frac{1}{T}\cdot u_r(\mathbf{x}, T), \quad (14)$$

where $u_r(\mathbf{x},T)$ can be derived from Eq. (6) or its simplification using Eq. (9) for $\tau = T$.

Therefore, a general component unavailability model can be derived using Eqs. (10)–(13) as follows:

$$u(\mathbf{x}) = u_r(\mathbf{x}) + u_t(\mathbf{x}) + u_m(\mathbf{x}) + u_c(\mathbf{x}) \quad (15)$$

which is needed to derive the risk function represented by $U(\mathbf{x})$ using Eq. (5).

As suggested earlier in this section, most of the parameters in the previous model may be adopted as decision variables in vector $\mathbf{x}$ for the optimization problem, as it is done for example in Refs. [10–11,24]. Here the interest is on optimizing TIs, represented through parameter T in the above expressions for a given component. Therefore, Eq. (15) can be reformulated to show the corresponding dependence on T as follows:

$$u(T) = f(\alpha, \frac{\beta}{T}, \gamma\cdot T) \quad (16)$$

where constants α, β, y and γ depend on the remaining parameters of the unavailability model associated with component reliability and downtime characteristics.

Eq. (16) is a practical but very simplified unavailability model as a consequence of the previous assumptions. A more realistic approach should extent this basic model in the PRA considering the effects of maintenance and working conditions on the component reliability characteristics, such as the ones presented in Refs. [39,40], which consider an age-dependent $\lambda(w)$ model, that in its simplest form in Ref. [10] yields to:

$$u(T) = f(\alpha, \frac{\beta}{T}, \gamma\cdot T, \eta\cdot T^2) \quad (17)$$

In general, more complicated dependencies exist in the previous unavailability model for both $u(T)$ and $U(T)$. In these cases analytical solutions cannot be derived and a numerical approach is required instead, i.e. Monte Carlo simulation.

In this paper, the standard PRA-based unavailability model of components suffices to demonstrate the TIs optimization approach based on GAs, so that, the model in Eq. (15) has been adopted for the sake of simplicity in the presentation. However, more detailed models are required in some cases, for example if one thinks about the idea of 'global optimization' of safety-related activities in a NPP (testing, maintenance, completion times, and so on), where all the relevant parameters have to be taken into account simultaneously for the decision-making. It is expected the maximum benefit of using GAs as optimizer in the solution of such complex problems.

2.3. Cost function

The relevant costs in analyzing test intervals optimization at system level involve the following contributions to the cost model of a given component:

$$c(\mathbf{x}) = c_t(\mathbf{x}) + c_m(\mathbf{x}) + c_c(\mathbf{x}) \quad (18)$$

where $c_t(\mathbf{x})$ represents a yearly cost contribution as consequence of the number of tests being performed on a particular component over a year period, $c_m(\mathbf{x})$ represents a yearly cost contribution as consequence of performing preventive maintenance on the component over a year

period and $c_c(\mathbf{x})$ represents a yearly cost contribution as consequence of performing corrective maintenance, the last devoted to repair component failures being discovered in a previous test.

Correspondingly, the cost model for the system of interest can be easily derived by summing up the corresponding cost contributions of the components that constitute the system, using Eq. (18) to yield:

$$C(\mathbf{x}) = \sum_i c_i(\mathbf{x}) \tag{19}$$

which will be used as the cost function in the constrained optimization problem.

Considering a component normally in standby, which undertakes surveillance testing with period T, preventive maintenance with period M and corrective maintenance to restore the operability of the component after a failure has been discovered during a test, the following basic cost contributions can be adopted [10,39]:

$$c_t(\mathbf{x}) = \frac{t}{T} \cdot c_{ht} \tag{20}$$

$$c_m(\mathbf{x}) = \frac{m}{M} \cdot c_{hm} \tag{21}$$

$$c_c(\mathbf{x}) = f_c(\mathbf{x}) \cdot d \cdot c_{hc} \tag{22}$$

where $f_c(\mathbf{x})$ can be obtained using Eq. (14) and the new constant parameters c_{ht}, c_{hm} and c_{hc} represent the hourly costs associated with conducting surveillance testing, preventive and corrective maintenance, respectively.

2.4. Constraints

In many cases, satisfying constraints is a difficult problem in itself. When constraints cannot be all simultaneously satisfied, the problem is often deemed to admit no solution. The number of constraints violated, and the extent to which each constraint is violated, needs to be considered in order to relax the preference constraints. In general, we can impose constraints over: (1) the objective functions; and (2) the values the decision variables in vector $\mathbf{x}$ can take.

In the first case, we apply constraints over one of the two objective functions, risk or cost function. These are referred as implicit constraints, where, for example, if the selected objective function to be minimized is the risk, $U(\mathbf{x})$, then the constraint is a restriction over the maximum allowed value to its corresponding cost. Alternatively, if the selected objective function to be minimized is the cost, $C(\mathbf{x})$, then the constraint is stated through the maximum allowed value for the risk.

In the second case, we impose constraints directly over the values the decision variables in vector $\mathbf{x}$ can take, which are referred as explicit constraints. Examples of this type of constraints in optimizing test intervals include limiting the TIs to take typified values, representing each one, for example, a day, a month or any other realistic period in the plant, instead of mathematical solutions that often are unpractical. Let us suppose a number E of periodically tested components in a system, with the corresponding test interval in the following vector of test intervals $\{T_1, \ldots, T_i, \ldots, T_E\}$. Then, we can formulate the above constraints as follows.

1. Overhaul time L, where:

$$T_i \leq L, \qquad \forall i \in [1, E] \tag{23}$$

2. Typified values (i.e. hours, days, months or years):

$$T_i = k \cdot T, \qquad k, T \in Z \tag{24}$$

Besides, we can also impose another explicit constraint of particular interest for TIs optimization at system or higher levels, adopting restrictions over the way in which test activities are undertaken. These restrictions are of concern to establishing test strategies, where a number of components are grouped to undertake testing under the same periodicity. That is, the test interval for all the components in a group takes the same or unified value under a given test strategy, which can be represented by a simultaneous, sequential or staggered testing. In addition, restrictions can also be imposed to the relationship between two different groups of components under the same test strategy. Let us suppose a number S of different groups of periodically tested components, each one grouping components that undertake testing under the same periodicity that corresponds to one of the following elements in vector of test intervals $\{T^1, \ldots, T^j, \ldots, T^S\}$. Then, we can formulate the above constraints as follows:

1. Same test strategy (i.e. simultaneous, sequential or staggered testing):

$$T_i^j = T^j, \quad \forall i \in j \text{ where } j \in [1, S] \;\&\; T^j \in Z \; \forall j. \tag{25}$$

2. Relationship between two different groups or strategies:

$$T^i = k \cdot T^j, \qquad k, T^i, T^j \in Z \;\; \forall i, j \in [1, S] \tag{26}$$

where it is satisfied that $S \leqslant E$.

The detailed description on how all these constraints enter the formulation of the optimization problem is provided in the next section along with the description of our SSGA as optimization method. However, in general, it can be said here that implicit constraints are considered as soft constraints to be handled using the penalization technique, while explicit constraints are considered as hard constraints, which are handled using the powerful SSGA encoding mechanism provided by GAlib.

3. A steady-state GA as optimization method

A steady-state GA (SSGA) has been customized to

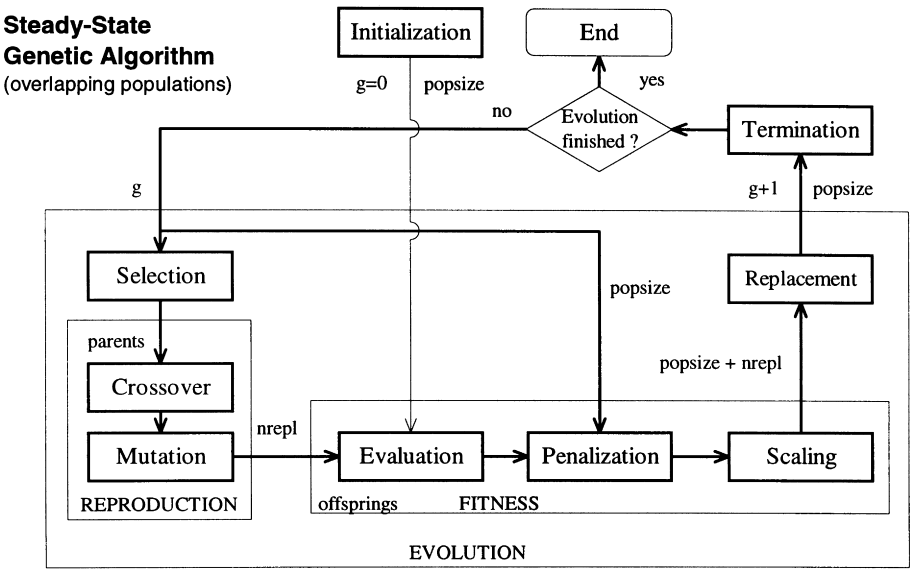


Fig. 1. Steady-state genetic algorithm scheme.

undertake the optimization problem using GAlib. Both the binary and the real-valued encoding of parameters is possible. The last has been preferred in this paper as problem formulation of the objective function and explicit constraints can be introduced in a very simple and natural way. In addition, new routines have been developed and implemented, mainly for handling implicit constraints using the penalization approach and also for verifying the GAs termination criteria. Next, the encoding mechanism and the fundamentals of the customized SSGA are presented.

3.1. Encoding

Traditionally, the variables of the optimization problem have been represented in GAs using binary strings [18,19,26]. In recent years, some authors have studied the convenience of using other alternatives, suggesting the utilization of more natural representations, such as the real-valued encoding [26,28,29]. These studies concluded that the selection of the encoding method depends on the features of the particular problem and there is no result that lead us to think that binary codification provides better results than the real one, or vice versa.

Test interval optimization normally faces real parameters, so that, the real-valued encoding mechanism has been selected using GAlib, as it provides a more natural representation of parameters. Real codification is performed by organizing the parameters to be optimized inside an array of real and independent variables **x**. It is in the definition of this array where the explicit constraints that apply to each real variable, x_i, can be established using GAlib.

Explicit constraints involve different aspects concerning, for example, with the type of variable, either discrete or continuous, and the range of valid values for each variable. In our case, parameters are represented in the appropriate vector **x** adopting discrete and integer variables, which range in a set of credible values that the test intervals could take in a real situation. The use of continuous variables could probably lead us to a better mathematical solution, but the values found for the optimized test intervals probably would not be suitable in real situations at NPPs.

Therefore, each array **x** will contain a possible solution to the optimization problem. In GAs terminology, each solution identifies an individual of the population of candidate solutions to the optimization problem, which is characterized by the particular genetic information stored in **x**. Correspondingly, this array is named genome (chromosome) where each variable x_i represents a gene of the genome [10].

GAs work in parallel with a population constituted by a number of different individuals. That is, a set of arrays of type **x** is considered, where each array stores a feasible solution to the optimization problem depending on the values adopted by the individual variables that encode the solution parameters.

3.2. Description of the customized SSGA

In the construction of our genetic algorithm (GA) we have used the GAlib library, since it presents a modular structure that allows the user to customize all, or part, of the algorithm stages. The GAlib library contains four flavors of GAs, among them we have chosen the steady-state genetic algorithm (SSGA) in which the initialization, evaluation, and termination stages have been customized, as it is presented in the following sections.

The main feature of the SSGA is the utilization of overlapping populations, as it can be observed in Fig. 1. The SSGA starts with an initial population of a given size. The

number of individuals that constitute this base population, denoted by popsize, is selected by the user. Each of these individuals is a possible solution to the optimization problem, which is given by the genetic information encoded in its corresponding genome **x**. This algorithm generates an auxiliary population, of size nrepl, constituted by the offspring obtained after the reproduction of certain individuals selected from the base population. Newly generated offspring is evaluated and then added to the base population. Each individual of the resulting population, composed by popsize + nrepl individuals, is penalized (if necessary) and then scaled to derive a ranking of individuals based on their fitness score. After scaling, the nrepl worst individuals in the ranking are removed in order to return the population to its original size (popsize). Therefore, after replacement, the best individuals remain in the new population constituting the new generation, generically denoted by $g + 1$, which descends from the previous one, g. The number of individuals to be replaced, nrepl, is fixed by the user and determines the amount of overlapping between two consecutive generations. One should realize the new offspring may, or not, be present in the new generation, depending on their fitness score. Once the new population has been generated, the algorithm checks if the termination criterion is achieved. If this criterion is not satisfied, then evolution continues to obtain a new generation as described previously, or the algorithm stops otherwise.

In this way, the SSGA accumulates information about an initially unknown search space and then exploits this information to guide subsequent searches into useful subspaces. Moreover, the SSGA has inherent parallel search properties since they work in a parallel way on a number of search points (individuals of the population) that are potential solutions to the optimization problem. This means that the SSGA has global scope over the search space so it is possible to get global optima.

3.2.1. Initialization

The first step in the evolution chain consists of the creation of an initial population ($g = 0$) of a certain size (popsize), which is not treated as the others, as it is directly evaluated as shown in Fig. 1.

The selection of a suitable number of individuals in the population, popsize, is an important aspect to be considered, since it will affect the behavior of the algorithm. Thus, with a small population, we take the risk of not covering the whole search space and the genetic algorithm could converge prematurely. On the contrary, working with too many individuals in the population could suppose an unnecessary computational cost [42]. Different experiments have been performed to obtain an appropriate population size, showing that in most cases a population with a number of individuals between 50 and 100 provides good results in optimization problems.

Usually, the initial population is generated using a random method, but it can also be obtained by applying different heuristic techniques, depending on the particular problem. In our case of application, several individuals of the initial population will correspond to the actual test intervals implemented in plant. This condition will also be used to establish the maximum value allowed for the risk and cost constraints, which will be obtained by evaluating the corresponding risk and cost models under the current test intervals. In this way, one can guarantee that at least the initial solution satisfies the constraints and, therefore, the problem admits solution in any case.

3.2.2. Selection

After initialization, the algorithm undertakes the evolutionary cycle according to the schematics of the SSGA shown in Fig. 1. Evolution continues in following generations ($g > 0$) with the selection of some individuals from the base population to be parents in the reproduction stage. The tournament selector has been chosen here from the available methods to select individuals in GAlib. This criterion is an extension of the roulette wheel method [26], which is a stochastic sampling method that picks the individuals by simulating the roulette wheel. In both criteria, each individual is assigned a probability of being selected based on its fitness score relative to the rest of the population. Thus, the better score the more likely an individual will be selected. The probability associated with an individual, i, is given by:

$$p(i) = \frac{\varphi(i)}{\sum_{i=1}^{\text{popsize}} \varphi(i)} \qquad i = 1, \ldots, \text{popsize} \tag{27}$$

where $\varphi(i)$ is the fitness score for that individual.

The tournament selector uses the roulette wheel to select a group of two, or more, individuals from the base population and then picks the one with the best score in this group. This process is repeated until the required number of selected individuals (nrepl) is reached. Therefore, following Nature's philosophy, the best individuals must reproduce more to create more offspring containing part of their genetic information. Thus, mainly the best solutions will prevail in the offspring and their genetic information will pass to the next generation.

3.2.3. Reproduction

The objective of reproduction is to create new individuals using genetic operators. A genetic algorithm uses, at least, two genetic operators for reproduction: (1) crossover, and (2) mutation [10].

The crossover operator combines two parents to obtain an offspring with genetic information similar to their parents. The one point crossover has been chosen from the crossover methods available in GAlib, which is a very simple method widely used that provides good results. This method consists first of selecting randomly a position in the genome of both parents by which each genome of the couple will be cut into two pieces. Then, we swap the two tails of the parents to get

the new two children (offspring). Usually, crossover is not applied to all the couples being selected for mating. A random choice is made depending on a probability of crossover provided by the user. As suggested in Ref. [18], recommended values for this probability range into the interval [0.6, 0.99].

Like in real life, we can introduce the mutation possibility over chromosome genes. This allows us to have a little random search where we can find improvements from generation to generation if mutation procedure yields good effects. The mutation operator is applied on an individual, altering its genetic information. The change in the individual genes provides a warranty that the algorithm will explore the whole search space. In our study, the flip mutation has been chosen from those methods available in GAlib. This method works by selecting randomly a position of the genome and then, based on the mutation probability, changing the value of the gene in that position by a different one that must satisfy the same explicit constraints. Therefore, not all the individuals are mutated, since only a part of them, parents or offspring, undergoes mutation according to the mutation probability provided by the user. A study performed by Bäck [18] shows that a value of $1/n$, with n being the length of the genome, is an appropriate value for the mutation probability under binary encoding and unimodal objective functions, which range in the interval $[1/n, 1]$ for multi-modal functions [22]. Although our study (real-valued encoding and multi-modal objective function) differs from the one preformed by Bäck, after several experiments we have found that a value of $1/n$, with n being the length of the array of variables, $\mathbf{x}$, also provides good results to our optimization problem.

3.2.4. Evaluation

After reproduction, the next step in evolution (Fig. 1) corresponds to the evaluation of the new individuals according to the objective function being adopted, which is represented here by $f(i)$ for the individual i. We will select either the risk or cost model as objective function depending on the particular optimization problem to be solved. In the first study, the risk model is taken as objective function and the cost is adopted as an implicit constraint. In the second one, the problem considers the cost model as objective function in the optimization process, adopting the risk as an implicit constraint.

3.2.5. Penalization

Once the values of the objective function for the new individuals have been calculated, the algorithm checks if the implicit constraints are satisfied, penalizing those individuals that violate one or more of such constraints. In our customized SSGA, the amount of penalization will depend on the degree of violation and the number of generations evolved according to an appropriate penalty function.

Penalty functions artificially create an unconstrained optimization problem, which cope with infeasible solutions by weighting a penalty based on judgement. In the literature, one can find different methods for penalizing the individuals that do not satisfy the constraints [26,29,30,32]. One of the families of penalty functions widely used is based on traditional calculus-based methods that is formulated as follows [30]:

$$P(\alpha, \beta, i) = K(g, \alpha)\cdot \mathrm{SVC}(\beta, i) \tag{28}$$

where the parameters α and β are defined by the user and $K(g, \alpha)$ is an α-function that establishes the pressure on infeasible individuals taking into account the number of generations evolved, g. Finally, $\mathrm{SVC}(\beta, i)$ is a β-function that represents the sum of violated constraints for a given individual i, which is formulated as follows:

$$\mathrm{SVC}(\beta, i) = \sum_j D_j^{\beta}(i) \tag{29}$$

The sum in Eq. (29) extends to the total number of constraints to which each individual, i, is subjected, where for a given constraint, j, the corresponding $D_j^{\beta}(i)$ is defined by the expression:

$$D_j^{\beta}(i) = \begin{cases} 0\,, & \text{if } g_j(i) < g_j^L \\ \left[\dfrac{g_j(i) - g_j^L}{g_j^L}\right]^{\beta}, & \text{if } g_j(i) \geq g_j^L \end{cases} \tag{30}$$

which represents the degree of violation associated with restriction j, where $g_j(i)$ represents the value achieved by the individual i with regard to constraint j, whereas g_j^L is the maximum value allowed for this constraint j.

There is another family of penalty functions, where their origin comes from simulated annealing, that also provides good results [30]. This family is defined as follows:

$$\mu(\alpha, \beta, i) = \exp[-P(\alpha, \beta, i)] \tag{31}$$

where $P(\alpha, \beta, i)$ is derived using Eqs. (28)–(30).

In our customized SSGA, the function used to penalize infeasible solutions is a variation of the family functions described by Eq. (31). This function introduces penalization relative to the worst score encountered in the population after the evaluation stage for the current generation, which also depends on the number of generations evolved, g. Thus, the penalized objective function for a given individual, i, is as follows:

$$\tilde{f}(i) = f(i) \pm [1 - \mu(\alpha, \beta, i)]\cdot f(w) \tag{32}$$

where $f(i)$ represents the raw score of the initial objective function for the individual i, and $f(w)$ is the corresponding score for the worst individual found in the population at current generation g. The sign of Eq. (32) differs whether the objective function is minimized or maximized. In the former case, the penalization is added to the initial objective function score, while it is subtracted otherwise. In both cases penalization alters the initial objective function score evaluated for each individual making it worse from

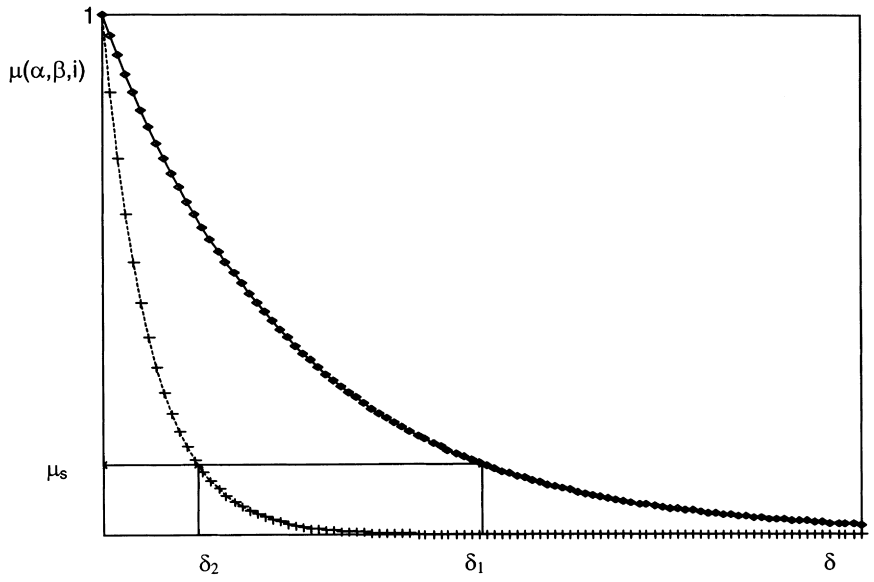


Fig. 2. Penalty function saturation.

the optimization point of view as long as it violates any constraint.

Several authors suggest that, provided that pressure parameters are appropriately chosen, a dynamic penalization is more efficient that a constant one, the former being dependent on the generations evolved, g [29,30]. Thus, at early generations it is more convenient to consider a relatively large tolerance, allowing a larger constraints violation by reducing penalization, as this extends the search space allowing unfeasible individuals that could lead up to feasible ones in a later generation. However, as the algorithm evolves the search space must be restricted to those individuals close to the optimum, which, in addition, have to satisfy all the constraints defined in the optimization problem. Correspondingly, the tolerance allowed has to be reduced as the algorithm evolves to reach a value close to zero after the evolution of a sufficiently large number of generations.

This feature is provided in this paper using an appropriate function $K(g, \alpha)$. In order to show how a self-adaptive penalization is achieved, let us consider the family of penalty functions $\mu(\alpha, \beta, i)$ derived by using Eqs. (28)–(31) and plotted in Fig. 2. It is assumed that the exponential term reaches saturation at a given value, named μ_s, which is very close to zero. Then, at the first generation, $g = 1$, an initial tolerance value δ_1 is admitted, which is reduced as the algorithm evolves until a certain generation, $g = \text{Gs}$, where a smaller tolerance value δ_{Gs} is allowed, which is less indulgent with the individuals that violate constraints. In Fig. 2, one can observe that a family of exponential functions is generated, $\mu(\alpha, \beta, i)$, ranging between the curves with exponents $K(1, \alpha)\cdot\delta^{\beta}$ and $K(Gs, \alpha)\cdot\delta^{\beta}$.

Therefore, $K(g, \alpha)$ determines the pressure on unfeasible individuals depending on the current generation, g, that is being evolved. The pressure on unfeasible individuals has to rise as the number of generations evolved increases. Different expressions for $K(g, \alpha)$ have been proposed [29,33]. In this paper, the function selected is given by:

$$K(g, \alpha) = K_1\cdot[(g - 1)\cdot p + 1]^{\alpha} \tag{33}$$

where g corresponds to the current generation number, and K_1 and p are the two parameters to be determined according to the desired values of δ_1 and δ_{Gs}, and the number of generations, Gs, for a given μ_s. Using Eqs. (28, 31 and 33) we can determine the expressions for K_1 and p. Thus, considering only one implicit constraint and using Eqs. (28) and (31), we have:

$$\mu(\alpha, \beta) = \exp[-K(\alpha,\ \beta)\cdot\delta^{\beta}] \tag{34}$$

and taking into account Eq. (33) for $g = 1$ ($\delta = \delta_1$) and $g = \text{Gs}$ ($\delta = \delta_{\text{Gs}}$) we obtain:

$$K_1 = \frac{\ln\left(\dfrac{1}{\mu_s}\right)}{\delta_1^{\beta}} \tag{35}$$

$$p = \frac{\left(\left(\dfrac{\delta_1}{\delta_{\text{Gs}}}\right)^{\frac{\beta}{\alpha}} - 1\right)}{\text{Gs} - 1} \tag{36}$$

Suitable values for δ_1, δ_{Gs}, Gs and μ_s are 0.01, 0.001, 1000 and 0.01, respectively. In addition, common values for the couple (α, β) include (0.5, 2), (1, 2) and (2, 2). However, a sensitivity study is required in order to select the appropriate parameters depending on the particular optimization problem.

Fig. 3 shows the different types of pressure on unfeasible individuals depending on α (0.5, 1, 2). In this figure, we can observe that for a given generation, g, the quadratic function ($\alpha = 2$) reaches a higher value of $(1 - \mu)$, which means that

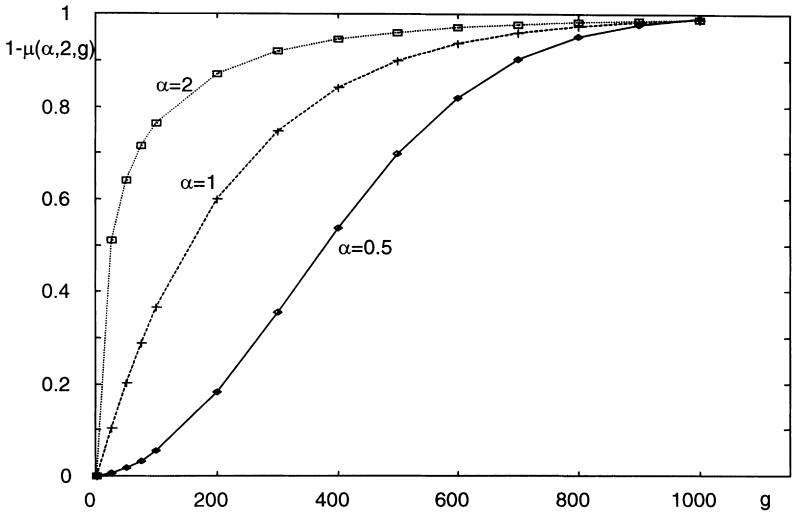


Fig. 3. Evolution of a α-set family of penalty functions with generation g.

unfeasible individuals are penalized stronger under this function. Therefore, one can control the pressure by selecting a certain α, for example lineal pressure with $g(\alpha = 1)$, strong at the beginning and weak at the end ($\alpha = 2$), or weak at the beginning and strong at the end ($\alpha = 0.5$).

3.2.6. Scaling

Once the score of each individual of the population, of size popsize + nrep, has been evaluated using the appropriate objective function and penalized, if necessary, the evolution continues towards the scaling stage as shown in Fig. 1. Scaling of scores is performed to avoid two problems that can appear in the evolution of GAs [26]. The first one comprises the appearance, at early generations, of few extraordinary individuals, named super-individuals, among other mediocre ones. This could lead after a few generations, if not avoided, to a situation in which the extraordinary individuals take over a significant portion of the population, leading the GA to premature convergence.

The second problem may occur late in a run when the average fitness score of the whole population is close to the fitness score of the best members in the population. If this situation is left alone, average members will probably get nearly the same number of copies in future generations and survival of the fittest necessary for improvement becomes a random walk among the mediocre individuals.

Using an appropriate scaling method it is possible to control the effects of selective pressure, therefore, softening them at the beginning of the run and hardening them at the end of the evolution. The linear scaling function has been chosen here from the different methods available to perform scaling in GAlib, which calculates the scaled fitness score for a given individual i, $\varphi(i)$, using the penalized objective function value in Eq. (32), $\tilde{f}(i)$, as follows:

$$\varphi(i) = a \cdot \tilde{f}(i) + b \tag{37}$$

where coefficients a and b are calculated for each generation.

3.2.7. Replacement

After scaling all the individuals in the population, the algorithm undergoes replacement as shown in Fig. 1. Several replacement methods are possible depending on the type of GA being used. For a SSGA, it only consists of performing a ranking of the total number of popsize + nrep individuals according to their fitness score given by Eq. (37) and, then, removing the nrepl worst fitted in order to restore the population to its initial size (popsize). Therefore, the new population includes the best individuals encountered after the algorithm has evolved from the previous generation, g, to the new generation, $g + 1$.

3.2.8. Termination

According to the schematics of the SSGA in Fig. 1, the evolutionary cycle is repeated through generations until the termination criterion is satisfied. The termination criterion specifies the conditions that must be satisfied to consider that the best solution has been found, or otherwise, it is accepted that the GA has evolved for a maximum number of generations.

One can find several termination criteria in the literature [16,17,32]. The simplest criterion, and also the most frequently used, consists of limiting the number of generations the GA has to evolve, named G, without checking whether a suitable solution has been found or not. It is assumed that after having evolved for G generations the algorithm should have found a good solution.

Other termination criteria are based on the search for population convergence. It has to be distinguished between two types of convergence: (1) fitness score convergence towards the same score; and (2) genetic convergence, or diversity, towards the same individual (the same genome

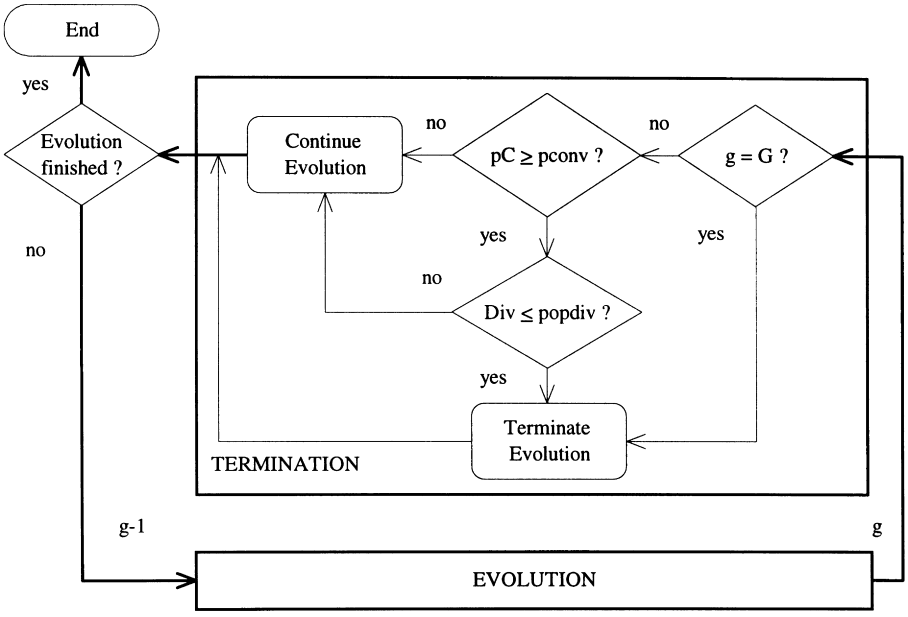


Fig. 4. Termination criterion scheme.

x). Both criteria may be necessary to admit that convergence is achieved.

As one can observe in Fig. 4, the termination criterion implemented in our customized SSGA determines whether evolution must finish by checking all three aspects:

1. Maximum number of generations (G).
2. Fitness score convergence (pconv).
3. Genetic convergence, or diversity (popdiv).

Firstly, the current generation, g, is compared with the maximum number of generations allowed, G, defined by the user. If the current generation equals G the algorithm stops. Otherwise, the termination criterion proceeds with checking convergence. Convergence is first checked based on the fitness score of population individuals in the current generation. In this paper optimization means minimizing either risk or cost functions, and consequently, the fitness score convergence criterion is formulated as follows:

$$\mathrm{pC} = 1 - \frac{|\varphi(m) - < \varphi(i) >|}{< \varphi(i) >} \geq \mathrm{pconv} \tag{38}$$

where $\varphi(m)$ is the minimum fitness score encountered, $\langle\varphi(i)\rangle$ is the averaged fitness score of the population in the current generation, and pconv is the convergence level required by the user.

As shown in Fig. 4, if the fitness score convergence criterion is not achieved, i.e. the condition pC ≥ pconv is not satisfied, the SSGA continues evolution. Otherwise, the algorithm proceeds with checking genetic convergence, or population diversity, analyzing whether the genes of all the population individuals in the current generation have achieved convergence. To decide when genetic convergence has been achieved, couples of individuals must be compared regarding the genetic information stored in their genes. This process is repeated for all the individuals (popsize) in the current generation in order to evaluate the population diversity as follows:

$$\mathrm{Div} = \frac{2}{\mathrm{popsize}\ (\mathrm{popsize} - 1)} \cdot \sum_{i=1}^{\mathrm{popsize}} \sum_{j=1}^{i-1} \frac{\sum_{k=1}^{L} \delta_{i_k,\, j_k}}{L} \leq \mathrm{popdiv} \tag{39}$$

where popdiv is the maximum diversity allowed by the user, i_k and j_k are the k–nth genes of the genomes i and j, respectively, L is the length of the genome and, in addition,

$$\delta_{i_k j_k} = \begin{cases} 0, & \text{if } |i_k - j_k| < \epsilon \\ 1, & \text{otherwise} \end{cases} \tag{40}$$

in which ϵ establishes the accuracy allowed when comparing genes. If the corresponding genes of two different individuals, i.e. genes in the same position, are identical, the value of δ_{i_k, j_k} from Eq. (40) equals to zero.

As shown in Fig. 4, if the genetic convergence criterion is not achieved, i.e. the condition Div ⩽ popdiv is not satisfied, the SSGA continues evolution, or otherwise the algorithm stops.

Evaluation of the population diversity is a very important topic, since it is possible to reach a situation with many different individuals in the population with similar fitness scores. This population would probably satisfy the fitness score convergence criterion, but with a so high diversity that it would not be possible to assure that the optimum solution has been reached. In fact, it could be expected that if evolution continues one of the individuals might lead to a better or optimum value. Controlling population diversity is of

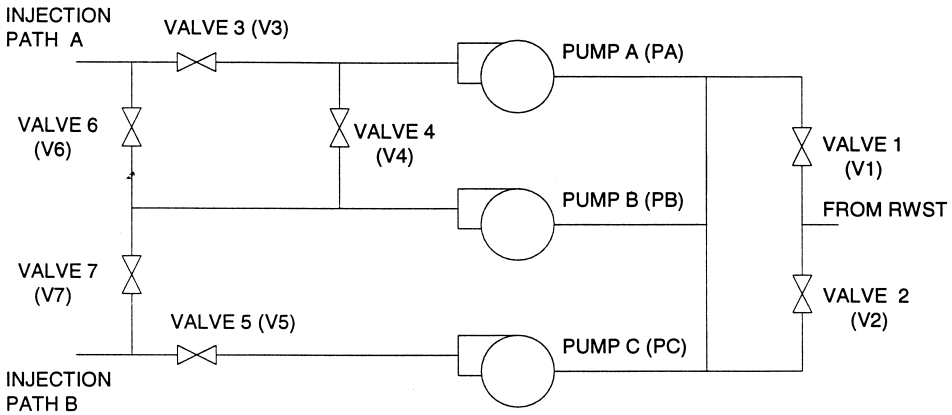


Fig. 5. HPIS system.

particular interest for unimodal functions, since at the end of the optimization process all the individuals of the population should equal the global optimum, and therefore, showing a population diversity equal to zero.

One advantage of our customized termination criterion is that the maximum number of generations allowed can be extremely high without necessarily supposing an excessive computational cost, since the algorithm will stop if fitness score and diversity convergence is achieved. However, it has to be taken into account that computing diversity is always a time consuming process, especially if the genome length is high. Even more, diversity convergence may not be achieved for the case of multi-modal functions that present local optima, making possible that different individuals, that is different genetic information, provide the same best fitness score (the best solution). Fortunately, by establishing a maximum number of generations allowed, G, one can guarantee that the SSGA will always stop.

4. Case of application

The methodology described in Sections 2 and 3 is applied herein for the constrained optimization of the test intervals of a safety-related system in a NPP. A simplified high-pressure injection system (HPIS) of a pressurized water reactor (PWR) is considered, which has been taken from Ref. [14]. This system is normally in stand-by and consists of three pumps and seven valves organized as shown in Fig. 5.

Under accidental conditions the HPIS can be used to remove heat from the reactor in those events in which steam generators are unavailable. For example, in case of a small-break loss-of-coolant accident (SLOCA) the HPIS safety function draws water from the refueling water storage tank (RWST) and discharges it into the cold legs of the reactor cooling system (RCS). Normally, pumps discharge into the injection paths A and B through valves 3 and 5, but crossover valves 4, 6 and 7 provide alternative flow paths in case of failure of the normal feed.

Table 1
Technical specifications for the HPIS

	Pump (P)	Valve (V)
STI (h)	2184	2184
AOT (h)	72	8

Table 1 shows typical AOT and STI requirements included into the HPIS Technical Specifications (TS). In addition, the relevant component unavailability and cost data for this case of application are adapted from Ref. [14] and are summarized in Table 2.

In this sample case, the system components have been grouped into three different test strategies. Strategy 1 applies to the valves that supply water from the RWST to the suction line of the three pumps, i.e. V1 and V2. Strategy 2 involves all three pumps, PA, PB and PC, and the valves located next to the discharges of pumps PA and PC, i.e. V3 and V5. Finally, crossover valves V4, V6 and V7 are included into the third strategy. Therefore, the following groups of test intervals have been considered:

$$T^1 = \left\{T^1_{V1}, T^1_{V2}\right\}$$

$$T^2 = \left\{T^2_{PA}, T^2_{PB}, T^2_{PC}, T^2_{V3}, T^2_{V5}\right\}$$

$$T^3 = \left\{\ T^3_{V4}, T^3_{V6}, T^3_{V7}\right\}$$

The imposed constraint assumes that all the components in the same group will adopt the same test interval after the optimization although this does not necessarily mean that all of them are tested simultaneously (i.e. sequential and staggered testing of redundant components in the same group is also possible adopting the same test frequency). In addition, it is imposed that test intervals of the three strategies must satisfy the following upper limits:

$T^1 \leq 8.760$ h
$T^2 \leq 87.600$ h
$T^3 \leq 876.000$ h

Table 2
Component unavailability and cost parameters

Unit	$\lambda(10^{-6}$/h)	$\rho(10^{-3})$	t (h)	d (h)	C_{ht} (\$/h)	C_{hc} (\$/h)
Valves (V)	5.83	1.82	0.75	2.6	20	15
Pumps (P)	3.89	0.53	4	24	20	15

Finally, the different test strategies must satisfy the following relationships in our particular case of application:

$$T^2 = k_2 \cdot T^1 \qquad T^3 = k_3 \cdot T^2$$

where k_2 and k_3 are integer variables that must range in the interval [1, 10]. T^1 is also an integer variable that must range in the interval [0, 8760].

Therefore, any solution to the optimization problem can be encoded using the following array of decision variables:

$$\mathbf{x} = \{T^1, k_2, k_3\}$$

where, as said above, T^1, k_2 and k_3 are integer variables in the optimization problem that must satisfy the previous explicit constraints.

Our case of study encompasses with two different optimization problems, considering the optimization of the risk function and adopting the cost function as an implicit constraint or vice versa.

In order to obtain the risk function, represented here by the system unavailability model, the system fault tree has been developed and then quantified using the MCS-based approach. Concerning the cost function, the corresponding model has been obtained considering only the costs associated with testing and repair.

The customized SSGA described in Section 3 has been used as optimizer, adopting the corresponding parameters that are summarized in Tables 3–5, which have been derived after running the algorithm for a number of trials. Tuning of parameters is a very important and problem-dependent task in order to guarantee the consistency of such a stochastic algorithm to find the same solution (the best one) after any trial.

In order to establish the risk or cost as implicit constraint, depending on the particular optimization problem, initial system unavailability and cost values have been derived by quantifying the corresponding model using the current test intervals (initial values of T^1, k_1 and k_2) shown in Table 6. Table 6 also contains the initial values achieved for both measures, $U(\mathbf{x})$ and $C(\mathbf{x})$, which have been used as upper limit constraints in the optimization problem.

Table 3
Parameters adopted for the genetic algorithm

Parameter	Values
Encoding mechanism	Real-valued (Integer)
Population size	100
Crossover probability	0.6
Mutation probability	0.3
Replacement Percentage	5
Maximum Number of generations	10 000
Convergence probability	0.99
Diversity	0.01

Table 4
Genetic operators

Operator	Method
Selection	Tournament Selector
Crossover	One point crossover
Mutation	Flip mutator
Scaling	Linear scaling

Figs. 6 and 7 summarize the evolution of the SSGA in minimizing the risk function adopting the initial cost as implicit constraint and the other explicit constraints described above for a typical run. Fig. 6 shows the evolution of the maximum, mean and minimum values of the fitness score through generations. Fig. 7 shows the evolution of both convergence criteria, i.e. the fitness score convergence and the genetic convergence (diversity) through generations. The results achieved for the optimized values of risk, the cost associated with that risk and the optimized decision variables are shown in Table 6. As one can realize, the optimized test intervals are extended with respect to their initial values except for the valves in the suction line that are reduced.

On the contrary, Figs. 8 and 9 summarize the evolution of the SSGA in minimizing the cost function adopting the initial risk as implicit constraint and the other explicit constraints described above for a typical run. The results achieved for the optimized values of cost, the risk associated with that cost and the optimized decision variables are also shown in Table 6. One can realize that the optimized test intervals are extended with respect to their initial values except for the valves in the suction line that are reduced again.

Finally, it must be noted that important reductions in both, risk and cost measures, have been achieved while all the explicit and implicit constraints are satisfied for the optimized test intervals in both cases.

5. Conclusions

Genetic Algorithms are very likely the most widely

Table 5
Parameters adopted for the penalization approach

Parameter	Values
α	1.0
β	2.0
Gs	10 000
δ_1	0.01
δ_{Gs}	0.001
μ_s	0.01

Table 6
Initial and optimized values

Variable[a]	Initial values	Optimized values Risk as Objective function	Optimized values Cost as Objective function
T_1(h)	2184	504	1512
k_2	1	6	5
k_3	1	2	2
$U(\mathbf{x})$	7.462×10^{-5} (constraint)	2.588×10^{-5}	7.268×10^{-5}
$C(\mathbf{x})$	1438.82 (constraint)	1424.34	565.257
CPU time (s)		263	225

[a] Decision variable vector $\mathbf{x} = \{T^1, k_2, k_3\}$, where each component represents an integer variable ranging in the intervals of allowed values: $T^1 \in [1,\ 8760]$, $k_2 \in [1,\ 10]$ and $k_3 \in [1,\ 10]$.

known type of Evolutionary Algorithms. However, EAs are essentially unconstrained search and optimization techniques. In fact, GAs are widely used now for unconstrained optimization problems with the general agreement about their fundamentals. Nevertheless, in the last few years, there has been a growing effort to apply GAs to general constrained optimization problems as most of engineering optimization problems often see their solution constrained by a number of restrictions imposed on the decision variables. In this paper, a Steady-State GA (SSGA) has been customized and successfully applied to perform the constrained optimization of test intervals at NPPs, where its capabilities of simplicity, flexibility, easy operation, minimal requirements and global perspective to find global optimum have been shown.

Regarding the SSGA implementation, the initial population is usually generated using a random method, but it can also be obtained by applying different heuristic techniques, depending on the particular problem. In this paper, several individuals of the initial population correspond to the actual test intervals implemented in plant, which yields to good results in our study. In this way, one can guarantee that at least the initial solution satisfies the constraints and, therefore, the problem admits solution in any case.

A generation-dependent dynamic penalization model, that incorporates self-adaptive pressure on unfeasible individuals, has been developed and successfully applied in this paper. Appropriate tuning of parameters of the customized penalization model is also another important factor in obtaining good performance of the SSGA for constrained optimization problems. Penalization indirectly controls the selection of parents that must lead to the best offspring by means of allowing unfeasible solutions that may lead to other feasible ones in later generations.

Many of the GA-based applications to engineering optimization problems only adopt the maximum number of generation as termination criterion. This criterion alone does not guarantee that an optimum solution will be achieved after a limited number of generations. However,

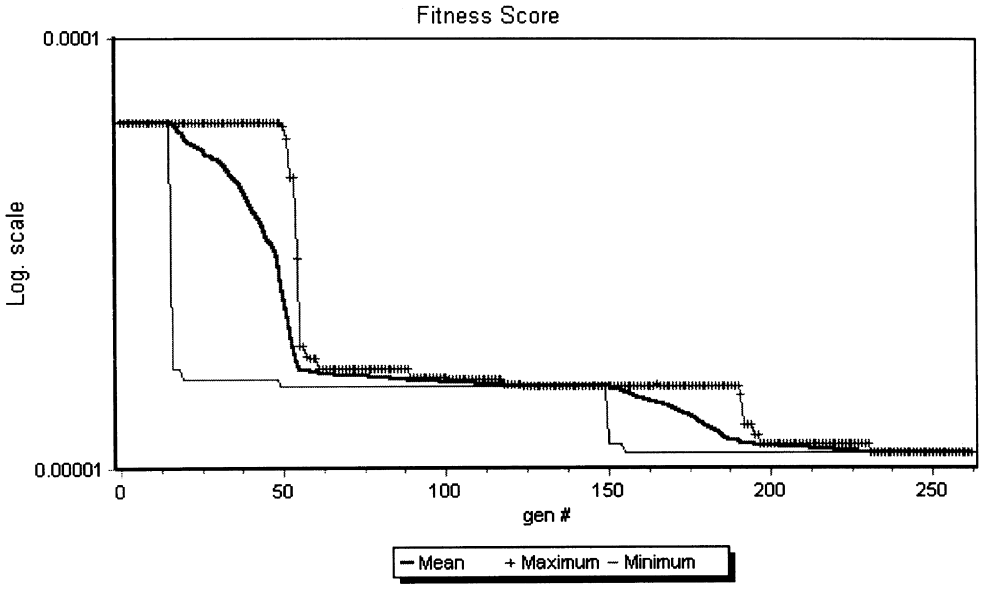


Fig. 6. Fitness score evolution in optimizing the risk function.

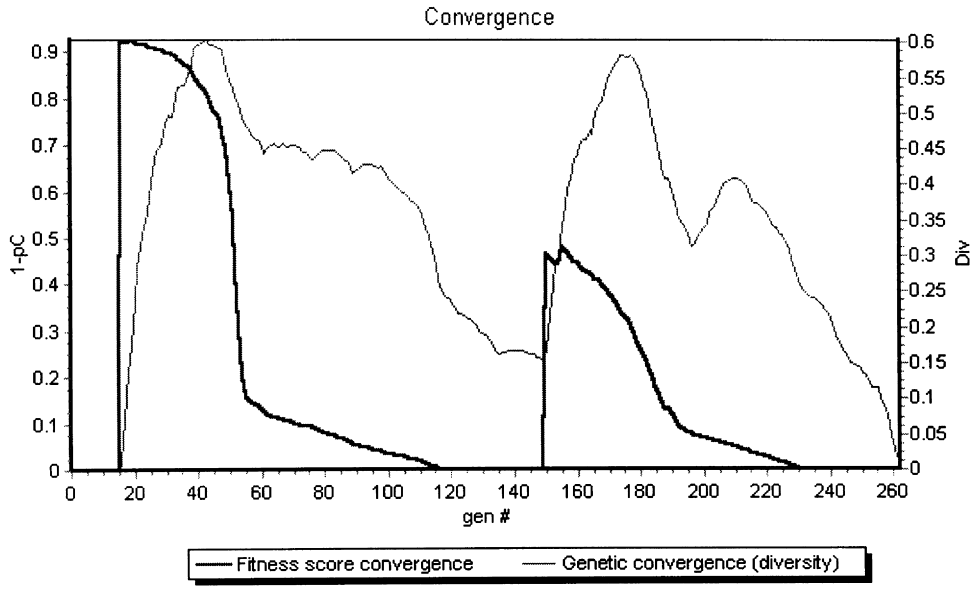


Fig. 7. Convergence evolution in optimizing the risk function.

by limiting the maximum number of generations one can assure that the GA will always finish reporting the best solution found until that moment.

On the contrary, often GA-based applications that consider convergence as termination criterion are only focused on the fitness score convergence, i.e. considering only the evolution of the fitness score shown in Figs. 6 and 8, or alternatively the variable pC plotted in Figs. 7 and 9. Looking at these figures, one can realize that fitness score convergence is achieved after few generations for both optimization problems ($\approx$ 150) and consequently the algorithm should stop evolution only if this convergence criterion was considered. Here, controlling diversity (variable Div plotted in Figs. 7 and 9) is a fundamental task, with similar results in both cases in order to avoid arriving to such an erroneous conclusion. Thus, diversity different from zero indicates that we have reached a situation with many different individuals in the population with the same or very similar fitness score, which may be interpreted as local optima in some cases or, otherwise, it is said that the SSGA presents premature convergence. However, as evolution continues through generations a better solution is found for both optimization problems, where, in addition, genetic convergence is satisfied (diversity is close to zero), showing uniformity among

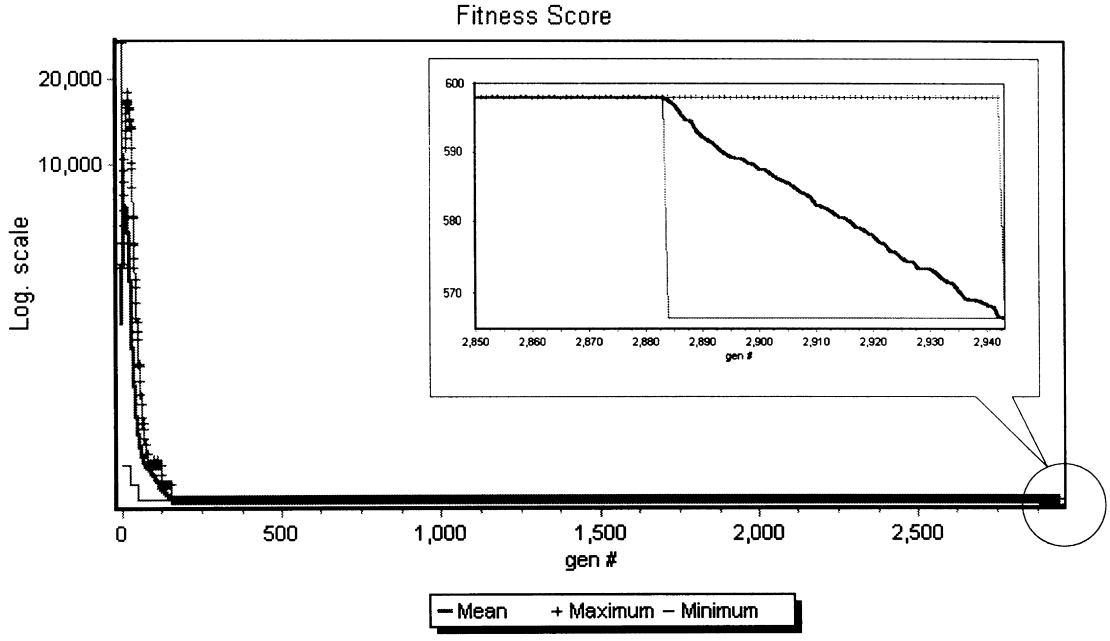


Fig. 8. Fitness score evolution in optimizing the cost function.

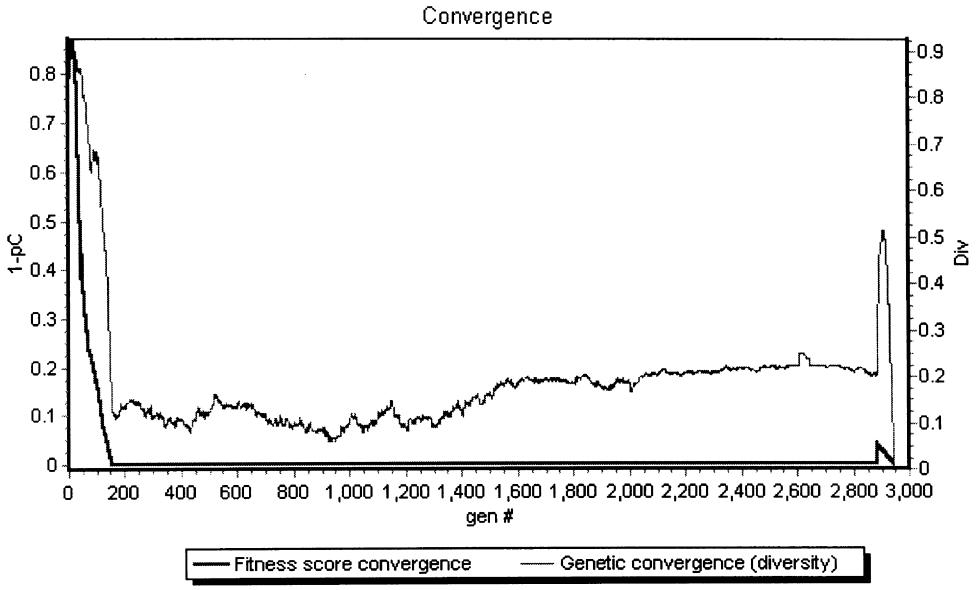


Fig. 9. Convergence evolution in optimizing the cost function.

the individuals in the final population, which may indicate that the global optimum has been found.

As one can observe in the previous Figs. 7 and 9, the convergence criteria are fully achieved in both cases, while the termination criterion based on the maximum number of generation does not apply in our case of study. This means that optimized values have been found for the decision variables in both cases, which also verify the imposed explicit and implicit constraints (see results in Table 6). In addition, since all the individuals in the final population have the same genetic information (diversity equals to zero), i.e. all of them hold the same value for the corresponding decision variables, one may believe that the global optimum under the above constraints has been found in both cases. However, only necessary conditions are satisfied, which are not sufficient to guarantee that the global optimum has been found. Further theoretical studies seem to be required in a future to find the sufficient conditions that demonstrate that the global optimum is achieved using GAs.

It is also important to notice that, in general, a larger number of generations is required in optimizing the cost function, which is justified as consequence of the different landscape to be explored by the SSGA algorithm in searching for the best solution depending on the particular objective function.

Regarding the optimized test intervals in this paper, all of them represent practical alternatives in both optimization problems, beyond of only mathematical solutions, which can be implemented in a NPP under a real situation. In addition, other constraints of concern to the NPP staff can be introduced in a very simple way using the methodology described in this paper.

In this paper, a standard PRA-based component unavailability model has been used to demonstrate the TIs optimization approach based on a SSGA. However, more detailed models may be required in some cases, for example if one thinks about the idea of 'global optimization' of safety-related activities in a NPP (surveillance testing, maintenance, completion times, IST, ISI and so on), where all the relevant parameters have to be taken into account simultaneously in the decision-making. It is expected the maximum benefit of using GAs as optimizer in the resolution of such complex problems.

References

[1] Vesely W, Apostolakis G. Developments in risk-informed decision-making for nuclear power plants. Reliability Engineering and System Safety 1999;63:223–4 Editorial.

[2] USNRC. Use of probabilistic risk assessment methods in nuclear activities: final policy statement. Federal Register, vol. 60, 16 August 1995. p. 42 622.

[3] USNRC Regulatory guide 1.174. An approach for using probabilistic risk assessment in risk-informed decisions on plant-specific changes to the current licensing basis, February 1998.

[4] Caruso M, Cheok M, Cunningham M, Holahan G, King T, Parry G, Ramey-Smith A, Rubin M, Thadani A. An approach for using risk assessment in risk-informed decisions on plant-specific changes to the licensing basis. Reliability Engineering and System Safety 1999;63:231–42.

[5] EPRI TR-105396, PSA Applications Guide. August 1995.

[6] CSN-UNESA, Documento básico para el proceso de realización y evaluación de las aplicaciones de los APS. In Spanish. Grupo Mixto para APS y Banco de Datos, Internal Report. Edicion 0, 11 November 1998.

[7] Vesely W. Principles of resource-effectiveness and regulatory-effectiveness for risk-informed applications: Reducing burdens by improving effectiveness. Reliability Engineering and System Safety 1999;63:283–92.

[8] USNRC Regulatory Guide 1.177. An approach for plant-specific changes, risk-informed decision-making: Technical Specifications. August 1998.

[9] UPV-CSN-UNESA, Proyecto de I + D para la realización de una

experiencia piloto de la aplicación de los APS a la modificación de las especificaciones técnicas. In Spanish, 11 November 1998.
[10] Muñoz A, Martorell S, Serradell V. Genetic algorithms in optimizing surveillance and maintenance of components. Reliability Engineering and System Safety 1997;57:107–20.
[11] Vaurio JK. Optimization of test and maintenance intervals based on risk and cost. Reliability Engineering and System Safety 1995;49:23–36.
[12] Muñoz A, Martorell S, Serradell V. Numerical absolute and constrained optimization of maintenance based on risk and cost criteria using genetic algorithms. In Proceedings of Advances in Safety and Reliability, ESREL'97, Lisboa 1997;3:1749–56.
[13] Fletcher R. Practical methods of optimization. New York: Wiley, 1987.
[14] Harunuzzaman M, Aldemir T. Optimization of standby safety system maintenance schedules in nuclear power plants. Nuclear Technology 1996;113:354–67.
[15] Righini R, Bottazzi A, Dubi A, Fichera C, Gandini A, Kladias N, Simonot H. A new Monte Carlo method for reliability centered maintenance improvement. Proceedings of Probabilistic Safety Assessment and Management, ESREL'96 1996;3:1745–58.
[16] Beasley D, Bull D, Martin R. An overview of genetic algorithms: Part 1. Fundamentals. University Computing 1993;15(2):58–69.
[17] Beasley D, Bull D, Martin R. An overview of genetic algorithms. Part 2. Research topics. University Computing 1993;15(4):170–81.
[18] Bäck T. Evolutionary algorithms in theory and practice. evolution strategies, evolutionary programming, genetic algorithms. New York: Oxford University Press, 1996.
[19] Holland J. Adaptation in natural and artificial systems. Ann Arbor: University of Michigan Press, 1975.
[20] De Jong K. An analysis of the behaviour of a class of genetic adaptive systems. PhD thesis, University of Michigan, 1975.
[21] Fogel D. An introduction to simulated evolutionary optimization. IEEE Transactions on Neural Networks 1994;15(1):3–14.
[22] Herrera F, Verdegay JL, editors. Genetic algorithms and soft computing Heidelberg: Physica-Verlag, 1996.
[23] Kumar A, Pathak R, Grupta Y. Genetic-algorithms-based reliability optimization for computer network expansion. IEEE Transactions on Reliability 1995;44(1):63–8.
[24] Painton L, Campbell J. Genetic algorithms in optimization of system reliability. IEEE Transactions on Reliability 1995;44(2): 172–8.
[25] Levitin G, Lisnianski A. Join redundancy and maintenance optimization for multistage series-parallel systems. Reliability Engineering and System Safety 1999;64:33–42.
[26] Goldberg DE. Genetic algorithms in search, optimization and machine learning. Reading, MA: Addison-Wesley, 1989.
[27] Davis L. Handbook of genetic algorithms. New York: Van Nostrand Reinholt, 1991.
[28] Fogel DB. Evolutionary computation. New York: IEEE Press, 1995.
[29] Michalewicz Z. Genetic algorithms + data structures = evolution programs. 3. Berlin: Springer, 1996.
[30] Joines JA, Houck CR. On the use of non-stationary penalty functions to solve nonlinear constrained optimization problems with GAs. Proceedings of the Evolutionary Computation Conference-Poster Sessions, part of the IEEE World Congress on Computational Intelligence, Orlando, 26–29 June 1994. p. 579–84.
[31] Michalewicz Z. A survey of constraint handling techniques in evolutionary computation methods. In: McDonnell JR, Reynolds RG, Fogel DB, editors. Proceedings of the Fourth International Conference on Evolutionary Programming, San Diego, CA, 1995. p. 135–55.
[32] Carlson S. A general method for handling constraints in genetic algorithms. University of Virginia, 1995.
[33] Carlson S, Shonkwiler R, Babar S, Aral M. Annealing a genetic algorithm over constraints. University of Virginia, 1995.
[34] Martorell S, Serradell V, Samanta PK. Improving allowed outage time and surveillance test interval requirements: a study of their interactions using probabilistic methods. Reliability Engineering and System Safety 1995;47:119–29.
[35] Leemis L. Reliability. probabilistic models and statistical methods. Englewood Cliffs, NJ: Prentice-Hall, 1995.
[36] Henley J, Kumamoto H. Reliability engineering and risk assessment. Englewood Cliffs, NJ: Prentice-Hall, 1961.
[37] Kim I, Martorell S, Vesely W, Samanta P. Quantitative evaluation of surveillance test intervals including test-caused risks. NUREG/CR-5775. February 1992.
[38] Kim I, Martorell S, Vesely W, Samanta P. Risk analysis of surveillance requirements including their adverse effects. Reliability Engineering and System Safety 1994;45:225–34.
[39] Martorell S, Muñoz A, Serradell V. Age-dependent models for evaluating risks and costs of surveillance and maintenance of components. IEEE Transactions on Reliability 1996;45(3):433–42.
[40] Martorell S, Sanchez A, Serradell V. Age-dependent reliability model considering effects of maintenance and working conditions. Reliability Engineering and System Safety 1999;64:19–31.
[41] Vaurio JK. Availability and cost functions for periodically inspected preventively maintained units. Reliability Engineering and System Safety 1999;63:133–40.
[42] Miller BL, Goldberg DE. Optimal sampling for genetic algorithms. Illigal report no. 96005, 1996.